ShopSmart test record - individual activity

Student: Paul Richard Calma
Date: 12 November 2026 (Asia/Manila)
Version: 1.0.0-quality-rc1 working tree, base commit ba45bbf
Execution: assisted automated tests in headless Microsoft Edge against local running services. No unobserved manual test results are claimed.

Baseline before fixes

The first npm run check completed successfully: frontend ESLint, backend ESLint, 8 frontend tests, 12 backend tests and the Vite production build. This result was observed in tool console output before edits; a complete baseline log was not saved. Full final output is retained separately. Automated success did not cover the assistant defects below.

PostgreSQL 18 was already running; npm run dev started frontend 5173 and API 3000. Initial API health returned 200 status ok. Six products existed, IDs 7-12; keyboard #8 already had stock 0. Database baseline contained **zero orders**. No seed/reset command was run during this activity. See before snapshot.

Implemented feature inventory and journey

Area	Present behavior
Frontend	Catalog, search, categories, sorting, details, memory cart, simulated checkout, assistant, error/loading states. Category/sort were inventoried in source but not separately exercised in the browser record.
API	GET /api/health, GET /api/products, GET /api/products/:id, POST /api/orders, POST /api/assistant/questions. Validation, rate limiting on assistant and atomic stock decrement are implemented.
Database	Prisma 6 PostgreSQL Product, Order, OrderItem; decimal prices; committed initial migration.
Authentication / roles	Absent; guest/owner/other-customer protected-record tests are N/A, not passed. No order-history GET route exists.
Live AI	Gemini integration present; an earlier run observed mode ai. The refreshed screenshot run returned mode rules with an explicit provider-unavailable notice. Catalog facts and source links remained correct.
Configuration	Frontend VITE_API_URL is public; database/provider settings remain on server.

Journey: browser -> catalog/search -> details -> cart -> simulated checkout -> API validation + stock transaction -> order confirmation -> assistant -> current product source. Failures must return control to the user.

Defects reproduced and fixed

D1 - unsupported assistant claims (high priority)

Starting state: catalog fixture Keyboard #1, PHP 1,850.50, stock 2, description Mechanical keyboard. Action: call answerQuestion with an injected draft selecting #1 and claiming a lifetime warranty and free worldwide delivery. Expected: only claims supported by catalog. Observed before: invented prose was returned with a real catalog source and mode ai. This reproduces without a live provider.

Fix: AI may select eligible IDs; displayed answer is composed only from selected catalog names/prices/stock. Arbitrary provider prose is not displayed. Two original claim-pattern tests alone could not prove semantic support. Repeat: regression and full journey passed. See failing regressions, passing regressions and backend/test/release-regressions.test.js.

D2 - budget centavos lost (medium priority)

Action: parse keyboard below PHP 1850.50, then rank a keyboard priced 1850.50. Expected: budget 1850.50 and eligible match. Observed: parser returned 1850, wrongly excluding a matching item. Fix: capture decimal portion and allow single-digit budgets. Repeat: 1850.50 includes fixture; 1850.49 excludes it; under 5 parses 5. Same before/after regression logs as D1.

D3 - database migration history mismatch (release blocker)

Starting state: database recorded 20260910093853_initial_schema; repository contained 20260910094941_initial_schema. prisma migrate status exited 1 and reported no common migration. Expected: matching committed history without resetting existing data. Compared SQL SHA-256 after LF normalization: 92d58d61b55594bb90289978ab052c252d182f774e5a329d465aaa2273cab1ad, exactly the stored applied checksum. Restored the recorded directory name and preserved SQL content; added LF policy. Database history was not rewritten. Status now up to date; db:deploy reports no pending migrations. See status, deployment, deployment after smoke.

D4 - current database password present in public examples

Configured-value scan found the current classroom password in README.md and backend/.env.example, while the built frontend was clean. Replaced public examples with REPLACE_WITH_LOCAL_PASSWORD; local .env was preserved. Expanded ignore rules for environment variants. See final secret scan. Scope is current submission files and bundle, not full Git history or an exhaustive third-party secret audit.

D5 - dependency advisories

Initial npm audit after browser-tool installation reported 13 vulnerabilities (5 moderate, 5 high, 3 critical). Applied compatible fixes, removed unused backend Vitest (backend uses node --test) and updated frontend Vitest within major version 5. Final audit reports zero. See before and after. All final tests/build were rerun after these changes.

Browser/API cases

Start the frontend and API, then run **npm run test:smoke** from a separate terminal. Requires Microsoft Edge, local test database and default ports 5173/3000 with 3001 free. For stable evidence capture, run **npm start --workspace backend** and **npm run dev --workspace frontend** in separate terminals; the API start command avoids file-watcher restarts. Each successful smoke run creates one simulated order and overwrites current evidence. Do not reseed to make a case pass.

The current demo uses **Alexandra Eala** as a sample name, with invented email **alexandra.eala@example.test** and address **123 Fictional Street, Demo City**. These are demonstration details, not a record of a real person's purchase. Items contain productId and quantity only. Exact requests and clicks are in scripts/release-smoke.cjs.

Final refreshed run: 11/12/2026, 2:02:27 AM (Asia/Manila). **11/11 passed**. Raw JSON / console. API screenshot index / unmodified API response records.

API screenshots show actual captured method, URL, status, request body and response JSON in a plain evidence viewer. They are not screenshots of Postman/DevTools or mock responses. No credentials or authorization headers are displayed.

HEALTH

- **Case and starting state:** Local API and PostgreSQL running
- **Action and data:** Open http://localhost:3000/api/health in browser
- **Expected:** 200 and status ok
- **Observed:** Browser displayed {"status":"ok"}; HTTP 200.
- **Decision:** PASS
- **Screenshots:** 11-api-health.

BROWSE

- **Case and starting state:** Existing catalog, fresh browser session
- **Action and data:** Open frontend; search keyboard; search zz-no-such-product-qa; clear search; open first Details
- **Expected:** Catalog, relevant search, useful empty state, and product detail
- **Observed:** 6 products loaded; keyboard found; absent query showed empty state; Wireless Mouse detail loaded.
- **Decision:** PASS
- **Screenshots:** 01-catalog, 02-no-results, 12-api-products, 13-api-product-search, 14-api-product-details.

CART

- **Case and starting state:** Product detail, empty memory cart
- **Action and data:** Add Wireless Mouse; increase to 2; decrease to 1; remove; add again; refresh cart
- **Expected:** Accurate totals; removal and intentional refresh reset
- **Observed:** Quantity 1 total PHP 650; quantity 2 total PHP 1300; decrease/remove worked; refresh cleared memory cart.
- **Decision:** PASS
- **Screenshots:** 03-cart.

INVALID-CHECKOUT

- **Case and starting state:** Cart with one product
- **Action and data:** Try empty form in browser; POST invalid email and short address directly
- **Expected:** Browser refuses; API returns 422 INVALID_ORDER
- **Observed:** Required field blocked browser submission; API HTTP 422 INVALID_ORDER for invalid email/address.
- **Decision:** PASS
- **Screenshots:** 16-api-invalid-order.

STOCK

- **Case and starting state:** Existing out-of-stock product or above-stock request
- **Action and data:** Inspect disabled purchase control; direct POST unavailable product or stock+1 units (max 20)
- **Expected:** Disabled button; API 409 INSUFFICIENT_STOCK; no order
- **Observed:** Compact Mechanical Keyboard stock 0; quantity 1 returned HTTP 409 INSUFFICIENT_STOCK: Only 0 unit(s) of Compact Mechanical Keyboard are available.
- **Decision:** PASS
- **Screenshots:** 04-stock, 17-api-stock-rejection.

CHECKOUT

- **Case and starting state:** One cart unit; no successful order in this run
- **Action and data:** Fill Alexandra Eala, alexandra.eala@example.test, 123 Fictional Street, Demo City; submit once
- **Expected:** One 201 order; server total equals catalog price; cart cleared; stock decrements once
- **Observed:** HTTP 201; order #3; total PHP 650; cart 0; product #7 stock 11 -> 10.
- **Decision:** PASS
- **Screenshots:** 05-order-confirmation, 18-api-order-created.

ASSISTANT

- **Case and starting state:** Assistant page; current catalog
- **Action and data:** Ask wireless mouse under PHP 700; inspect answer and click catalog source
- **Expected:** Eligible current product, correct price, linked record; controlled fallback allowed
- **Observed:** HTTP 200; mode rules; Based on the current ShopSmart catalog, consider: Wireless Mouse (₱650, 10 in stock).; source link opened current product.
- **Decision:** PASS
- **Screenshots:** 06-assistant, 19-api-assistant.

NO-MATCH

- **Case and starting state:** Assistant page
- **Action and data:** Ask Do you sell a drone?
- **Expected:** Honest no match, no sources/recommendations, loading ends
- **Observed:** HTTP 200 no-match; honest message; sources and recommendations empty.
- **Decision:** PASS
- **Screenshots:** 07-no-match, 20-api-assistant-no-match.

API-FAILURE

- **Case and starting state:** Assistant page; browser API requests can be intercepted
- **Action and data:** Abort assistant request in browser, submit mouse; then abort product requests and open home
- **Expected:** Useful error, enabled submit button, no endless loading; catalog offers backend guidance
- **Observed:** Assistant displayed Failed to fetch and re-enabled submit; catalog ended loading and displayed backend guidance. Browser route abort simulated API outage; live server remained running.
- **Decision:** PASS
- **Screenshots:** 08-assistant-network-failure, 09-catalog-network-failure.

PROVIDER-FAILURE

- **Case and starting state:** Separate local API on 3001 with empty provider key, same database
- **Action and data:** Start API with GEMINI_API_KEY empty; route browser assistant request to that API; ask mouse
- **Expected:** 200 rules fallback, notice, source links, loading ends
- **Observed:** Real API with missing provider key returned HTTP 200 rules fallback, catalog sources and visible notice. No key was modified on disk.
- **Decision:** PASS
- **Screenshots:** 10-provider-fallback, 22-api-provider-fallback.

API-ENDPOINTS

- **Case and starting state:** Local API and existing catalog
- **Action and data:** GET search, product detail, invalid product ID, missing product; POST invalid assistant question
- **Expected:** HTTP 200, 200, 400, 404 and 422 with captured response screenshots
- **Observed:** Search/detail returned 200; invalid product ID 400; missing product 404; invalid question 422. Thirteen API request/response screenshots and raw JSON captured across this run.
- **Decision:** PASS
- **Screenshots:** 15-api-invalid-product-id, 21-api-invalid-question, 23-api-product-not-found.

Retained failure and unresolved limitations

The first evidence-refresh attempt was interrupted before checkout by an API file-watcher restart: the stock request received ECONNREFUSED. No new order was created by that attempt. Retained interrupted run JSON and log. Restarted the API with **npm start --workspace backend** (without watching files), then all 11 cases passed. The latest assistant response used its controlled rules fallback; successful live-AI operation is an earlier observation, not a claim about this refreshed run.

The first browser run passed health/browse/cart/invalid checkout but timed out waiting for the out-of-stock detail page. It was not declared passed: initial JSON, initial log. Direct API and separate browser diagnosis worked; the complete repeat after restarting updated dependencies passed. Root cause of that one timeout was not established. Review repeat stability if it recurs.

Network failure is simulated by browser route abort (server stays running). Missing provider configuration is exercised against an isolated real API on 3001 with an empty inherited key, without editing .env. Existing service unit test also exercises a throwing provider. No claim is made that an actual paid-provider service outage occurred.

Cart reload reset is intentional and documented. Keyword matching, no authentication, no real payments and no idempotent retry guarantee remain classroom limitations. Staging is blocked by lack of approved hosting/database/access setup; local release is complete. No live staging URL or access test exists.

Final release checks

Check	Observed result	Decision/evidence
npm run check	Both lint checks, 8 frontend + 14 backend tests, Vite build succeeded	PASS; full output
Local smoke	11 cases completed, including controlled failures and API endpoint evidence	PASS; browser records above
Database preservation	Original run: 0 -> 1 order (#2). Evidence refresh: 1 -> 2 orders (#2 and #3), preserving #2; mouse stock 11 -> 10; six products. A subsequent db:deploy preserved both orders. No reseed.	PASS; refresh before, refresh after, migration verification. Only demo orders were exercised.
Secrets	No prohibited .env files in current submission list; configured secret values not present in submission text/build	See final secrets-check.json; historic Git secrets not audited
Privacy	Fictional demo checkout details only; screenshots contain no credentials	PASS for recorded demo
Private staging	No approved platform or deployed URLs	NOT RUN; local-only release

Individual record

Paul Richard Calma is the sole project contributor for this activity. Assisted automated evidence and source changes were prepared in this working tree. Before submission, personally repeat and explain one test (suggested: checkout/stock), the assistant grounding contribution and why migrate deploy differs from destructive reseeding. This record does not attribute tool-executed clicks to unobserved personal manual work.